

Shape-Aware Sparse Traffic-Matrix Analytics for Reproducible GraphChallenge Anonymized Network Sensing

Qifan Chen

University of California, Santa Barbara
qifan_chen@ucsb.edu

Abstract—The MIT/IEEE/Amazon GraphChallenge Anonymized Network Sensing track asks participants to construct and analyze anonymized source-to-destination traffic matrices from packet-derived data. We present a commodity-CPU reproducibility toolkit whose central empirical finding is that candidate-tracking capacity, not Count-Min sketch width, bounds heavy-hitter discovery on diffuse traffic: widening the sketch from 512 to 32,768 does not recover a single missed heavy hitter, while raising candidate capacity from 64 to 8,192 lifts diffuse-regime top-20 recall from 0 to 1.00. We validate against the official challenge data products: byte-range prefixes of the official Random PCAP dataset (40,000,000 packets parsed in seconds) and the official GraphBLAS matrices, which our parser reproduces exactly (64 of 64 windows, 8,388,608 pairs). The dataset is an extreme all-tie diffuse regime in which every packet is a unique source-destination pair, and the early-stream selector and a conservative sufficient-condition safety screen both correctly flag fixed-candidate sketching as unsafe while recommending the measured-fastest exact construction; a SuiteSparse:GraphBLAS baseline builds the same aggregated matrix within 25% of our SciPy kernel while its native 2^{32} -index hypersparse mode eliminates label factorization entirely. On synthetic regimes up to 5,000,000 edge records, sparse direct construction wins on diffuse traffic while pandas pre-aggregation wins on highly duplicated hotspot traffic; a 50,000-record prefix correctly predicts the exact-method winner in all tested regimes, and the screen’s SpaceSaving retention bound predicts the candidate-capacity scale that the empirical sweep independently confirms. As a toolkit capability, controlled time-bin experiments additionally identify volume, scanner, concentrated-destination, and distributed low-rate scan anomalies. The contribution is not full-capture throughput, but a transparent, reproducible, student-oriented bridge between official GraphChallenge materials and commodity sparse-matrix experimentation.

I. INTRODUCTION

Network sensing tasks often begin with very large streams of packet or flow records. The Anonymized Network Sensing Graph Challenge frames these records as sparse source-to-destination traffic matrices suitable for graph and linear-algebra analysis [1]. Official GraphChallenge submissions are paper-based and reward measured innovation in graph analysis software, hardware, algorithms, and systems [2], [3]. This submission targets a complementary point in the GraphChallenge design space: not maximum throughput, but a reproducible, inspectable, low-dependency baseline that makes traffic-shape dependence and approximation risk measurable.

To be explicit about what kind of paper this is: it is a reproducibility-and-diagnostics contribution built around one central empirical finding, not a performance record. The central finding is that candidate-tracking capacity, not Count-Min sketch width, bounds heavy-hitter discovery on diffuse traffic. The contributions are layered around that finding. The *diagnostic layer* – the primary contribution – turns the finding into online decision rules: an early-stream method selector and a conservative sufficient-condition sketch-safety screen, both computable from a stream prefix. The *validation layer* supports the diagnostics with multi-regime synthetic benchmarks over equivalence-checked exact baselines, with byte-range prefixes of the official Random PCAP data product fetched and parsed by a small reproducible script and analyzed across multiple stream segments, with an exact cross-check against the official GraphBLAS matrices, and with a SuiteSparse:GraphBLAS construction baseline. The *toolkit layer* – included as capability, not as a headline claim – adds time-bin anomaly summaries, official-format compatibility checks, and scripts that regenerate every table and figure in this paper.

II. RELATED WORK AND POSITIONING

GraphChallenge traffic-matrix tasks are closely aligned with sparse linear algebra. GraphBLAS defines graph analytics in terms of sparse matrix operations [4], and SuiteSparse:GraphBLAS shows how such ideas can be implemented efficiently [5]. Recent reference-implementation work for GraphChallenge emphasizes improving official pipeline clarity, adaptability, and performance [6]. Our work is complementary: it does not replace a high-performance reference implementation, but focuses on a student-facing CPU toolkit with synthetic regime controls, streaming failure diagnostics, temporal anomaly summaries, and online method selection.

Network measurement has also long used sketches and heavy-hitter algorithms for high-rate streams, including Count-Min Sketch [7], Misra-Gries summaries [8], SpaceSaving/top- k stream tracking [9], and sketching systems for network measurements [10]. Traffic anomaly studies show that feature distributions and traffic shape can be as important as aggregate volume [11]. This package connects those ideas to the GraphChallenge setting while keeping the claim boundary conservative.

A. Student Innovation

The student innovation is not a new hardware record or a claim of full CAIDA-scale throughput. It is a low-barrier reproducibility package for commodity CPUs: deterministic data generation, exact command-line workflows, transparent pandas and Python baselines, multi-regime sparse traffic-matrix benchmarks, streaming/sketch analytics, time-bin anomaly summaries, and interpretable outputs that expose heavy hitters, source and destination structure, traffic concentration, and temporal bursts. The implementation is intentionally not optimized with GPU, C++, or distributed GraphBLAS, making it a baseline rather than a performance ceiling.

III. APPROACH

The input format is an anonymized edge table with source identifier, destination identifier, and a numeric traffic weight such as bytes or packets. The pipeline factorizes source and destination labels, constructs a SciPy coordinate matrix, converts it to compressed sparse row format, and sums duplicate source-destination pairs. This yields a sparse matrix $A \in \mathbb{R}^{m \times n}$ where A_{ij} is the aggregate traffic from source i to destination j .

The implementation then computes analytics intended to be useful for network sensing:

- matrix size, nonzeros, density, and total traffic;
- top- k heavy hitter source-destination pairs;
- source and destination degree and strength distributions;
- entropy of source strength, destination strength, and edge weights;
- a largest singular value as a lightweight spectral summary;
- a Gini coefficient of nonzero edge weights as a concentration statistic.

The repository includes three exact construction methods. The primary method builds a sparse matrix directly from edge records. A pandas baseline first groups by source and destination. A pure Python dictionary baseline aggregates pairs with a counter-like loop for small inputs. All exact methods are checked for output equivalence on the same synthetic inputs.

The new streaming path processes edge records sequentially. It keeps exact source and destination strength summaries, a fixed-size Count-Min Sketch for approximate edge weights [7], and a bounded candidate tracker inspired by streaming heavy-hitter algorithms [8], with lazy-deletion heap eviction so that even all-unique streams cost $O(\log c)$ per record. The streaming path is intentionally evaluated as an online approximation rather than a faster replacement for optimized SciPy sparse construction. Its main diagnostic is top- k heavy-hitter recall against the exact sparse matrix, which isolates candidate discovery from weight estimation and exposes the capacity bound reported in Section VII.

As a toolkit capability, the time-bin path groups the same edge records by a supplied `time_bin` field, computes per-bin volume, nonzeros, entropy, Gini, and top-share statistics, and

scores bins with a robust positive-deviation summary; temporal aggregation can expose burst, scan, or concentration patterns that disappear in a single summed matrix.

Finally, the online selector uses only an early stream prefix to compute duplication rate, estimated nonzero growth, source and destination entropy, edge Gini, top-edge share, and top source/destination concentration. It applies the rule table below and is meant to make traffic-shape dependence visible, not to replace profiling.

Algorithm 1: early-stream selector. Given a prefix, compute duplication rate d , nnz growth g , edge Gini G , top-edge share e , and top source/destination share s . If $d \geq 0.55$ or $G \geq 0.82$ or $e \geq 0.01$ or $s \geq 0.05$, recommend pandas pre-aggregation and mark sketch heavy hitters as plausible for concentrated traffic. Else if $d \leq 0.20$ and $g \geq 0.80$, recommend sparse direct construction and mark fixed-candidate sketches unsafe for top- k discovery. Otherwise, recommend sparse direct construction and profiling before using sketches. These thresholds are empirical and chosen only to separate the synthetic regimes used in this submission; they are not claimed to be universal.

IV. CONSERVATIVE SCREENING CONDITIONS

The safety screen adds conservative sufficient conditions to the empirical selector rule. We state its components and assumptions explicitly: it is a prefix-computed screen, not a distribution-free guarantee about the full stream.

For exact construction, let N be edge records, U unique source-destination pairs, and $\rho = U/N$. A first-order model $T_{\text{sparse}} = \alpha_s N + \beta_s U$, $T_{\text{pandas}} = \alpha_p N + \beta_p U$ predicts that pandas pre-aggregation wins only below a shape threshold in ρ ; the threshold is hardware-dependent, so this is intuition, not a law.

For sketch safety, the candidate tracker is SpaceSaving with weighted updates [9]: an arriving tracked key adds its weight; an untracked key replaces a minimum-count key and inherits that count plus its weight. By the standard SpaceSaving/Misra-Gries analysis [8], [9], every tracked count overestimates its key's true weight by at most $W/(c+1)$ for stream weight W and capacity c , and every key with true weight above $W/(c+1)$ is tracked. Let w_k be the k th largest aggregated prefix edge weight and $\Delta_k = w_k - w_{k+1}$ the ranking gap; we define top- k identity only when $\Delta_k > 0$, and the screen refuses when $\Delta_k = 0$ (ties), as on the official prefix below.

The screen assumes: (A1) *prefix representativeness* – weight shares and rank order measured on the prefix carry over to the stream (checked empirically across eight disjoint official-capture segments in Section VII, but not guaranteed); (A2) Count-Min with width m , depth d overestimates any queried weight by at most ϵW , $\epsilon = e/m$, with probability at least $1 - e^{-d}$ per query [7]. Under (A1)–(A2), three sufficient conditions follow. *Retention*: $c \geq W/w_k - 1$ ensures every true top- k pair is tracked. *Ranking*: $\Delta_k > 2\epsilon W$ ensures Count-Min estimates preserve the top- k boundary among tracked keys. *Combined*: the implemented screen requires the stricter $\Delta_k > 2(\epsilon W + W/(c+1))$, which additionally covers rankings read from tracker counts rather than sketch estimates. Failure

of the screen does not prove sketch failure; satisfaction certifies top- k retention and separation under the stated assumptions only.

V. REPRODUCIBILITY PACKAGE

The package is intentionally small. The main scripts are:

- setup, sample-generation, and sparse-analysis scripts;
- benchmark and figure scripts for exact, streaming, sensitivity, selector, and certified-selector experiments;
- a time-bin anomaly script for controlled network-sensing scenarios;
- an official-data prefix fetcher that Range-downloads the first N compressed bytes of the official `pcap.zst`, stream-decompresses the truncated `zstd` frame, parses the fixed-size packet records, and emits an anonymized `src,dst,bytes` edge table with a provenance manifest;
- an optional reference wrapper and a local GraphChallenge-paper-style formula comparison.

The synthetic generator is not a substitute for CAIDA-scale data. It provides four regimes: `hotspot_zipf`, `community_bursty`, `scanner_fanout`, and `uniform_sparse`. It also exposes controlled knobs for edge count, source and destination counts, target nonzeros or density, duplication rate, Zipf skew, community structure, scanner fanout, and temporal bursts. These controls make the local benchmark runnable without protected downloads or multi-gigabyte packet captures while still testing multiple sparse traffic shapes.

VI. EXPERIMENTS

Experiments were run on a MacBook Air with Apple M3 CPU, 8 cores, 16 GB RAM, macOS 26.5.1 arm64, Python 3.9.6, NumPy 2.0.2 with Apple Accelerate BLAS/LAPACK, SciPy 1.13.1, pandas 2.3.3, matplotlib 3.9.4, and zstandard 0.25.0. No GPU, distributed runtime, or manual BLAS thread pinning is used.

The official-data experiments use the two official challenge data products published in the GraphChallenge S3 bucket: the Random PCAP (13.9 GB `pcap.zst`) and the Random GraphBLAS matrices (8.6 GB tar); these are the organizer-generated products in official formats, while the CAIDA-telescope-derived variant is access-controlled and not used here. An HTTP Range request fetches the first 256 MiB (later extended to 2 GiB) of the `pcap.zst`, the truncated `zstd` frame is stream-decompressed, and 5,000,000 (respectively 40,000,000) fixed-size packet records (RAW-IP link type, 40-byte IPv4/TCP headers) are parsed with a vectorized NumPy path in under five seconds per 5,000,000 packets. Identifiers are the challenge-provided anonymized addresses used as opaque integer labels; no re-anonymization is applied. A provenance manifest records the URL, byte range, parser path, packet counts, and output checksum. One 67 MB inner tar of the GraphBLAS product (the first 2^{23} -packet segment) is fetched the same way for the exact cross-check, deserialized with `python-graphblas` on SuiteSparse:GraphBLAS.

Traffic-matrix construction and summary runtime

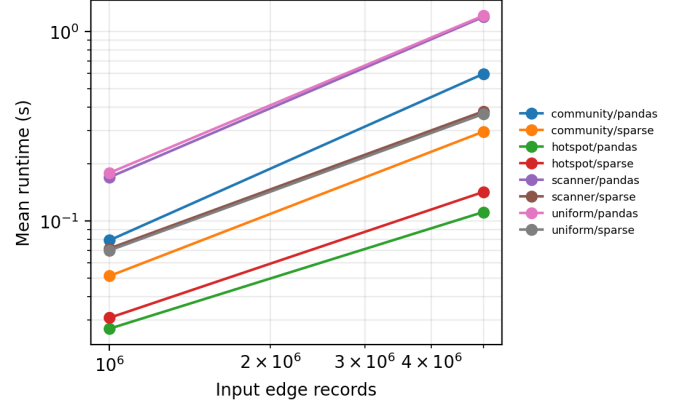


Fig. 1. Large commodity-CPU benchmark up to 5,000,000 edge records across four synthetic regimes. The winner changes with duplication and nnz growth, so a single construction strategy is not uniformly best.

The small benchmark uses generated edge tables at 5,000, 25,000, and 100,000 records, with three repeats and three exact methods. The large benchmark uses 1,000,000 and 5,000,000 records, one repeat, and two scalable exact methods. Reported construction and analytics runtimes exclude synthetic data generation and disk I/O unless otherwise stated; the input edge table is already resident in memory. The benchmark also records edges per second, nonzeros per input edge, and process peak RSS divided by input edges. The streaming benchmark uses 25,000, 100,000, and 500,000 records. Sensitivity studies vary sketch width and candidate capacity at 100,000 records. The method-selector and certified-selector experiments use the first 50,000 records of each generated 5,000,000-edge stream. Time-bin experiments inject volume burst, scanner fanout, concentrated-destination, and distributed low-rate scan events into synthetic bins. A small official-format smoke test writes and reloads a GraphBLAS-style Matrix Market sparse coordinate matrix and checks shape, nonzeros, and total weight against the edge-table pipeline. Commands are exposed through the repository Makefile.

The key correctness check is output equivalence. For every benchmark row, the matrix produced by the method under test is compared with the sparse direct matrix. Total traffic is also compared. A benchmark row is marked valid only when both checks pass.

VII. RESULTS

Figure 1 shows the large benchmark. All numbers and figures in this section were generated on the machine described in Section VI, and the repository Makefile regenerates every table and figure from saved CSV outputs.

Table I reports the 5,000,000-edge large benchmark. The regimes differ substantially: hotspot traffic has 222,097 nonzeros and high concentration, while uniform sparse traffic has 4,997,173 nonzeros and density 0.0011. Sparse direct construction is faster than pandas groupby in community, scanner, and uniform regimes. In the hotspot regime, pandas groupby

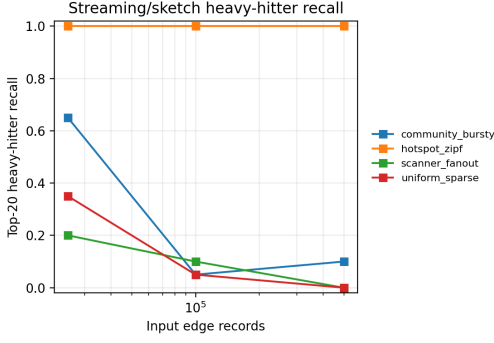


Fig. 2. Streaming/sketch top-20 heavy-hitter recall. The same fixed-size sketch is reliable for concentrated hotspot traffic but fails to discover diffuse heavy hitters, showing that sketch width alone is insufficient without adequate candidate tracking.

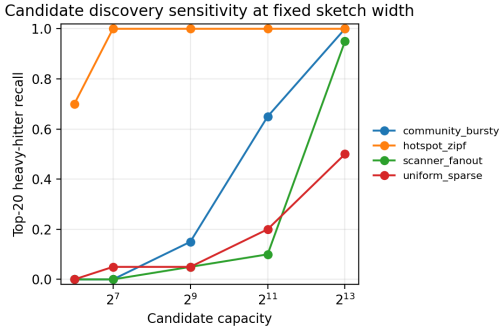


Fig. 3. Candidate-capacity sensitivity at fixed sketch width. Candidate capacity, rather than sketch width, is the limiting factor for diffuse heavy-hitter discovery.

is faster on this machine, which is a useful negative result: the best construction path depends on traffic shape and duplication rate. Throughput scales nearly linearly from 1,000,000 to 5,000,000 edges (sparse direct moves from 14.1–32.7 to 13.3–35.2 million edges per second across regimes), so the reported rates are not a small-input artifact. The table reports total measured construction plus analytics time and throughput for in-memory edge tables.

Table II summarizes the streaming/sketch result at 500,000 edges. The Count-Min Sketch uses 327,680 bytes. It recovers all top-20 heavy hitters in hotspot traffic with median relative error below 0.001, but recall falls to 0.10 for community traffic and 0 for scanner and uniform regimes. A width sweep showed that increasing sketch width from 512 to 32,768 reduces hotspot median weight error from 0.0405 to 0 but does not improve diffuse-regime recall at fixed candidate capacity. Figure 3 shows the missing factor: increasing candidate capacity from 64 to 8,192 raises community recall from 0 to 1.00 and scanner recall from 0 to 0.95. With the lazy-deletion heap tracker, that capacity increase costs almost nothing at run time – the 100,000-edge sweep moves from 0.62 s at capacity 64 to 0.71–0.91 s at capacity 8,192 – so candidate capacity is a memory knob, not a latency penalty, and there is little reason

TABLE I
LARGE BENCHMARK AT 5,000,000 INPUT EDGES (MINIMUM OF THREE REPEATS; STANDARD DEVIATION AT MOST 0.02 S PER CELL). RUNTIME EXCLUDES SYNTHETIC GENERATION AND DISK I/O.

Regime	nnz/edge	Sparse	Sp. M/s	pandas	pd. M/s
Hotspot	0.044	0.142	35.2	0.110	45.3
Community	0.451	0.285	17.6	0.590	8.5
Scanner	0.992	0.377	13.3	1.189	4.2
Uniform	0.999	0.366	13.7	1.205	4.1

TABLE II
STREAMING/SKETCH HEAVY-HITTER RESULT AT 500,000 EDGES.

Regime	Stream Time	Recall	Median Err.
Hotspot	3.45	1.00	0.0003
Community	3.24	0.10	1.0000
Scanner	3.42	0.00	1.0000
Uniform	3.36	0.00	1.0000

to starve it. In this Python implementation, streaming is still slower than SciPy sparse construction, so its value is bounded-memory online processing and a clear approximation baseline, not raw speed.

Table III reports the online method-selector experiment. From only the first 50,000 records, the selector identifies the hotspot regime as high-duplication and concentrated, recommends pandas pre-aggregation, and matches the saved 5,000,000-edge winner. For community, scanner, and uniform regimes, nonzeros grow almost one-for-one with the stream and the selector recommends sparse direct construction, again matching the saved winner. This result is useful because it turns the benchmark into an online decision rule with visible failure boundaries: sketches are plausible for concentrated heavy hitters, but diffuse traffic should default to exact sparse construction or use much larger candidate sets.

A. Official-Dataset Validation

The candidate-capacity finding and the online diagnostics were checked on the official challenge data products. Table IV summarizes the experiment on the 5,000,000-packet prefix of the official Random PCAP. The prefix is an extreme diffuse regime: every packet is a unique source-destination pair (duplication rate 0), there are 4,997,115 unique anonymized sources, and every packet is a 40-byte header-only record, so all edge weights are equal – consistent with the randomized generation the organizers describe. On this input the early-stream selector, using only the first 50,000 records, recommends sparse direct construction and flags fixed-candidate sketching as unsafe; the measured construction winner is sparse direct (2.52 s versus 5.15 s for pandas groupby, about 2.0×10^6 edges per second), so the recommendation matches. The safety screen withholds certification because the prefix top- k ranking gap is exactly zero. The capacity sweep confirms the diagnosis from the other direction: strict-identity top-20 recall is exactly zero at capacities 64 through 8,192, while tie-aware recall (counting any equal-weight candidate as a match) is trivially

TABLE III
EARLY-STREAM METHOD SELECTOR AT 50,000 OBSERVED RECORDS,
COMPARED WITH 5,000,000-EDGE EXACT-METHOD WINNERS.

Regime	Dup.	Top edge	Rec.	Winner
Hotspot	0.775	0.0948	pandas	pandas
Community	0.010	0.0007	sparse	sparse
Scanner	0.000	0.0003	sparse	sparse
Uniform	0.000	0.0003	sparse	sparse

1.0 – the divergence between the two definitions is itself the diagnosis, since *top- k identity* carries no information in an all-tie regime. Raising capacity from 64 to 8,192 changes the streaming pass from 33.7 s to only 35.0 s with the heap-based candidate tracker, so capacity is operationally cheap; it simply has nothing to recover here. This is the complementary case to the synthetic diffuse regimes: there, capacity rescues recall because heavy hitters exist but are hard to discover; here, the selector and screen correctly report that sketch-based *top- k* is ill-posed. Distinguishing these two situations online, from a 50,000-record prefix, is exactly what the screen is for. To address prefix representativeness (Assumption A1), we extended the fetch to a 2 GiB range (40,000,000 packets) and repeated the analysis on eight disjoint 5,000,000-packet segments: all eight yield duplication rate 0, nonzero growth 1.0, the same `sparse_direct` recommendation, and the same zero-gap screen refusal, so the diagnosis is stable across every segment we can reach, not an artifact of the first window.

Two further checks tie the toolkit to the official GraphBLAS ecosystem. First, an exact cross-check: the official GraphBLAS product stores one $2^{32} \times 2^{32}$ UINT32 matrix per 2^{17} -packet window; deserializing the first 2^{23} -packet segment (64 matrices) and comparing coordinate sets against our pcap-parsed edge table yields an exact match in 64 of 64 windows (8,388,608 pairs; the official matrices index addresses in host byte order, ours in network order, a bijective relabeling the comparison accounts for). Our parser therefore reproduces the official matrix pipeline on this segment, without running the official code. Second, a SuiteSparse:GraphBLAS construction baseline on the same 5,000,000 official-prefix edges: GraphBLAS `from_coo` with duplicate-summing builds the compact aggregated matrix in 0.086–0.088 s versus 0.106 s for the SciPy kernel, so the transparent SciPy path is within about 25% of the optimized library on the aggregation kernel itself. Label factorization, shared by both compact paths, costs 2.1 s and dominates; GraphBLAS can instead build directly in the native $2^{32} \times 2^{32}$ anonymized-address space (hypersparse) in 0.086 s, eliminating factorization entirely – an architectural advantage of the official GraphBLAS representation that SciPy CSR cannot replicate, since its row-pointer array alone would need tens of gigabytes.

Table V reports compatibility validation rather than a speed contest. The Matrix Market smoke test reloads a GraphBLAS-style sparse coordinate matrix and exactly matches the edge-table sparse matrix on shape, nonzeros, and total weight. The formula check compares our pipeline with a local implemen-

TABLE IV
OFFICIAL RANDOM PCAP PREFIX EXPERIMENT (5,000,000 PACKETS
PARSED FROM A 256 MiB BYTE-RANGE REQUEST IN UNDER FIVE
SECONDS).

Quantity	Value
Packets parsed	5,000,000
Unique src-dst pairs	5,000,000
Duplication rate	0.0
Prefix nnz per edge	1.0
Sparse direct construction (s)	2.52
pandas groupby construction (s)	5.15
Measured winner	<code>sparse_direct</code>
Selector recommendation	<code>sparse_direct</code> (match)
Safety-screen decision	not certified (gap = 0)
Recall@20 strict / tie-aware	0.00 / 1.00

TABLE V
COMPATIBILITY VALIDATION FOR OFFICIAL-FORMAT AND
REFERENCE-FORMULA PATHS.

Check	Input	nnz	Weight	Status
Matrix reload	10k MM	4,747	66.1M	match
Formula scalars	100k edge	75,952	648.3M	match
Official .grb	2^{23} pkt	8.39M	8.39M	64/64 exact

tation of GraphChallenge-paper-style sparse sensing formulas on a 100,000-edge community sample. This is not the full official GraphChallenge reference repository, but it verifies that core scalar quantities agree before performance claims are interpreted.

Finally, the time-bin capability: in all four controlled scenarios (volume burst, scanner fanout, concentrated destination, distributed low-rate scan), the injected bin is the top-ranked bin by the robust anomaly score, and a different feature explains each event type. We report this as a toolkit capability rather than a core contribution; per-scenario curves and numbers are in the repository figures and CSVs.

The selector separates exact-method choice from heuristic sketch use: pandas pre-aggregation is recommended only when early traffic is highly duplicated and concentrated, while fixed-candidate sketching is marked unsafe for all three diffuse regimes even when exact sparse construction remains fast.

Table VI reports the safety screen of Section IV from the same 50,000-record prefix under two sketch budgets. Under the default width 8,192 and candidate capacity 512, no regime satisfies the sufficient conditions. The screen is not a permanent refusal: raising the budget to width 16,384 and candidate capacity 5,000 (a 655 KB sketch) satisfies the combined condition for the hotspot regime with margin $+5.6 \times 10^{-5}$, while the diffuse regimes remain uncertified. Notably, the rigorous retention bound now predicts the capacity scale that the empirical sweep independently confirms: the screen computes $c_{\min} \approx 2,000$ for the community regime, where measured recall reaches 0.65 at capacity 2,048 and 1.00 at 8,192, and $c_{\min} \approx 5,800$ for the scanner regime, where recall reaches 0.95 only at 8,192. The screen therefore tells an operator both when sketch safety is not proven and

TABLE VI

SAFETY SCREEN FROM A 50,000-RECORD PREFIX UNDER TWO BUDGETS: DEFAULT (WIDTH 8,192, CAPACITY 512) AND TUNED (WIDTH 16,384, CAPACITY 5,000). $c_{\min}$ IS THE SPACESAVING RETENTION BOUND $\lceil W/w_{20} \rceil - 1$; $\text{WIDTH}_{\min}$ IS THE COUNT-MIN WIDTH REQUIRED BY THE GAP CONDITION.

Regime	p_{20}	$c_{\min}$	$\text{Width}_{\min}$	Default	Tuned
Hotspot	.352	173	6.9k	no	yes
Community	.011	1,990	2.4M	no	no
Scanner	.004	5,839	2.5M	no	no
Uniform	.004	6,654	24.3M	no	no

what budget would prove it. Applied to the official-capture prefix, it withholds certification because the ranking gap is zero, matching the all-tie shape reported above.

The current benchmark supports three conservative observations. First, all local exact construction paths produce equivalent aggregate matrices on the tested inputs, which is necessary before performance comparisons are meaningful. Second, the sparse path provides a concise implementation that naturally exposes interpretable sensing summaries without densifying the traffic matrix. Third, multi-regime, online, and time-bin results matter: a method or sketch that looks best on high-duplication hotspot traffic may not dominate when nnz approaches the edge count, early stream statistics can steer exact-method choice, and temporal bursts can be easier to explain through bin-level summaries than through a single aggregate matrix.

VIII. LIMITATIONS

This submission deliberately avoids overclaiming. The experiments combine controlled synthetic regimes with byte-range prefixes (up to 40,000,000 packets) of the official Random PCAP data product and an exact cross-check against the official GraphBLAS matrices. These validate parsing, selector behavior, safety-screen behavior, and capacity sensitivity on organizer-published official data, but they are not a full 2^{30} -packet run, the composition of a prefix may differ from the full product (the eight-segment stability check in Section VII mitigates but does not eliminate this), and the Random products are organizer-generated synthetic traffic – the CAIDA-telescope-derived variant is access-controlled and was not used, so no claim is made about real telescope traffic. The memory measurement is process-level and should be interpreted as a coarse operational signal. The runtime benchmark excludes disk I/O and synthetic generation; the official-prefix fetch, decompression, and parse times are measured separately and recorded in the committed provenance manifest. A full same-machine comparison against the official reference workflow remains future work. Finally, the synthetic generator is designed for reproducibility and sparse-regime coverage, not as a validated traffic model.

IX. CONCLUSION

We presented a shape- and time-aware, reproducible Python toolkit for anonymized network sensing with exact sparse, streaming/sketch, online selector, conservative safety-screen,

and time-bin analytics. The central empirical result is that candidate-tracking capacity, not sketch width, bounds heavy-hitter discovery on diffuse traffic: width sweeps reduce weight error without recovering a single missed heavy hitter, while capacity sweeps move diffuse recall from zero to near one – at the capacity scale that the screen’s SpaceSaving retention bound independently predicts. Around this result, the toolkit demonstrates that traffic regime changes the right method: sparse direct construction is usually faster than pandas groupby, pandas wins on highly duplicated hotspot traffic, early-stream features can recommend the observed exact-method winner on the tested regimes, and the screen identifies both when sketch safety is not proven and what sketch budget would prove it. These effects were confirmed on the official Random PCAP data product, where the selector and screen correctly identify an extreme all-tie diffuse regime across eight disjoint 5,000,000-packet segments, and the parser reproduces the official GraphBLAS matrices exactly on the first 2^{23} -packet segment. The time-bin capability adds a complementary, secondary lesson: temporal summaries can expose burst, scanner, concentrated-destination, and distributed scan behavior in a form that is easy to inspect. These failure cases and diagnostics are part of the contribution because they make approximation risk visible to student entrants. The package therefore serves as a conservative bridge between official GraphChallenge materials and commodity-CPU experimentation with sparse matrices, sketches, temporal signals, online method choice, and interpretable network-sensing summaries.

REFERENCES

- [1] “Anonymized network sensing graph challenge,” arXiv:2409.08115, 2024.
- [2] “MIT/IEEE/Amazon GraphChallenge,” <https://graphchallenge.mit.edu/>, accessed 2026-06-29.
- [3] “GraphChallenge Submission Instructions,” <https://graphchallenge.mit.edu/submit>, accessed 2026-06-29.
- [4] A. Buluç, T. Mattson, S. McMillan, J. Moreira, and C. Yang, “The graphblas c api specification,” in *IEEE International Parallel and Distributed Processing Symposium Workshops*, 2017.
- [5] T. A. Davis, “Algorithm 1000: Suitesparse:graphblas: Graph algorithms in the language of sparse linear algebra,” *ACM Transactions on Mathematical Software*, 2019.
- [6] I. Voloshchuk, H. Jananthan, C. Byun, and J. Kepner, “Improving the graph challenge reference implementation,” arXiv:2601.00974, 2026.
- [7] G. Cormode and S. Muthukrishnan, “An improved data stream summary: The count-min sketch and its applications,” *Journal of Algorithms*, 2005.
- [8] J. Misra and D. Gries, “Finding repeated elements,” *Science of Computer Programming*, 1982.
- [9] A. Metwally, D. Agrawal, and A. El Abbadi, “Efficient computation of frequent and top-k elements in data streams,” in *International Conference on Database Theory*, 2005.
- [10] A. Kumar, J. Xu, J. Wang, O. Spatscheck, and L. Li, “Sketching data streams for network measurements,” in *Proceedings of the ACM SIGCOMM Conference*, 2004.
- [11] A. Lakhina, M. Crovella, and C. Diot, “Mining anomalies using traffic feature distributions,” in *Proceedings of the ACM SIGCOMM Conference*, 2005.